

What Makes a Good LLM Agent for Real-world Penetration Testing?

Gelei Deng¹, Yi Liu¹, Yuekang Li², Ruozhao Yang³, Xiaofei Xie³,
Jie Zhang⁴, Han Qiu⁵, Tianwei Zhang¹

¹Nanyang Technological University, ²University of New South Wales, ³Singapore Management University,
⁴CFAR, A*STAR, Singapore, ⁵Tsinghua University

Abstract

LLM-based agents show promise for automating penetration testing, yet the reported performance varies widely across systems and benchmarks. We analyze 28 LLM-based penetration testing systems and evaluate five representative implementations across three benchmarks of increasing complexity. Our analysis reveals two distinct failure modes: *Type A failures* stem from capability gaps (missing tools, inadequate prompts) that engineering readily addresses, while *Type B failures* persist regardless of tooling due to planning and state management limitations. We show that Type B failures share a root cause that is largely invariant to the underlying LLM: agents lack real-time task difficulty estimation. As a result, agents misallocate effort, over-commit to low-value branches, and exhaust context before completing attack chains.

Based on this insight, we present PENTESTGPT v2, a penetration testing agent that couples strong tooling with difficulty-aware planning. A Tool and Skill Layer eliminates Type A failures through typed interfaces and retrieval-augmented knowledge. A Task Difficulty Assessment (TDA) mechanism addresses Type B failures by estimating tractability through four measurable dimensions (horizon estimation, evidence confidence, context load, and historical success) and uses these estimates to guide exploration-exploitation decisions within an Evidence-Guided Attack Tree Search (EGATS) framework. PENTESTGPT v2 achieves up to 91% task completion on CTF benchmarks with frontier models (39 to 49% relative improvement over baselines) and compromises 4 of 5 hosts on the GOAD Active Directory environment versus 2 by prior systems. These results show that difficulty-aware planning yields consistent end-to-end gains across models and addresses a limitation that model scaling alone does not eliminate.

1 Introduction

Penetration testing is essential for assessing organizational security, yet the demand for skilled practitioners far exceeds

supply. The ISC2 Cybersecurity Workforce Study estimates a global shortfall of 4.7 million cybersecurity professionals [14]. This gap, together with the labor-intensive nature of manual testing, has driven interest in large language model (LLM)-based automation.

Recent systems report strong results on benchmarks such as Capture-the-Flag challenges and Hack The Box (HTB) environments [8, 17, 19, 30, 32], and emerging work has demonstrated real-world impact, including the discovery of exploitable vulnerabilities in production software [10, 13]. However, reported task completion rates range from single digits under naive prompting to 40–80% with more sophisticated architectures [9, 20], raising a central question: *what drives these performance differences, and what limitations remain?*

To answer this question, we conduct a systematic analysis of 28 LLM-based penetration testing systems and evaluate five representative solutions across three benchmarks of increasing complexity. Our analysis yields two findings. First, existing systems are optimized to address the limitations of specific LLMs. For example, context summarization and RAG-augmented tooling are designed to compensate for transient LLM constraints of limited context windows and poor tool knowledge. Benefits brought by these designs quickly diminish as models improve: performance gaps across solutions compress by over half when backbone models upgrade from GPT-4o to GPT-5. Second, failures partition into two categories: *Type A failures* (capability gaps) stem from missing tools and knowledge addressable through engineering, while *Type B failures* (complexity barriers) persist regardless of tooling due to planning and state management limitations. Existing systems predominantly target Type A failures, achieving strong results on simple tasks but failing on multi-step scenarios where Type B failures dominate. This indicates that the architectures of existing penetration testing systems are not designed to complement the improvements of LLMs. Their contributions erode as models advance, rather than compounding with improved capabilities.

We trace Type B failures to a missing capability: *existing*

penetration testing agent designs cannot assess task difficulty in real time. This manifests in several ways: agents commit prematurely to unproductive branches because they cannot estimate whether a path requires 3 or 30 steps; they fail to transition from reconnaissance to exploitation because they lack metrics for evidence sufficiency; they experience context forgetting because they do not monitor context consumption. Human pentesters handle these problems through intuition built from experience. LLM agents lack equivalent mechanisms for difficulty-aware decision making. We validate this diagnosis through controlled evaluation: augmenting agents with difficulty assessment reduces the Type B failure rate from 58% to 27% while Type A rate remains unchanged, confirming that this enhancement addresses the root cause.

We present PENTESTGPT v2, designed around these two findings. To eliminate Type A failures, an extensible *Tool and Skill Layer* provides typed interfaces for 38 security tools and skill compositions that encode expert attack patterns. To address Type B failures, we introduce penetration testing *Task Difficulty Assessment* (TDA), a mechanism that estimates task tractability through four measurable dimensions: horizon estimation, evidence confidence, context load, and historical success rate. TDA is integrated into an *Evidence-Guided Attack Tree Search* algorithm that guides exploration-exploitation decisions and prunes branches when paths become intractable. With these mechanisms, PENTESTGPT v2 dynamically pivots between attack paths based on real-time difficulty signals. It abandons unproductive branches before they exhaust the context budget and commits to exploitation only when evidence confidence justifies the investment. A retrieval-augmented *Memory Subsystem* maintains structured state external to the LLM context, which prevents the context forgetting that derails extended attack campaigns.

We evaluate PENTESTGPT v2 across three benchmarks at different levels of realism, from CTF challenges to enterprise Active Directory environments. On XBOW [2] (104 web security tasks), PENTESTGPT v2 achieves 91% peak task completion (89% mean) with Claude Opus 4.5, a 49% relative improvement over the best baseline (61%). On the PentestGPT [8] Benchmark (13 HTB/VulnHub machines), PENTESTGPT v2 roots 12 of 13 machines, solving Hard-rated targets where baselines become stuck at initial steps. On GOAD (5-host Active Directory environment), PENTESTGPT v2 compromises 4 of 5 hosts compared to at most 2 for prior systems, with successful lateral movement and credential chaining across domain boundaries. Ablation studies confirm that each component contributes distinctly: the Tool Layer dominates on short-horizon tasks, while TDA-EGATS and Memory provide the gains on multi-step scenarios.

Despite these results, hard challenges remain. Our evaluation shows that novel exploitation requiring creative reasoning, adversarial environments with deceptive defenses, and extended multi-week campaigns exceed current LLM capabilities. These limitations suggest that fully autonomous penetra-

tion testing remains distant. We discuss these boundaries and propose evaluation methodologies that distinguish tractable from intractable challenges, so that the community can focus effort where architectural innovation is most likely to help.

In summary, we make the following contributions:

- **Systematic analysis of LLM agent failures (§3).** We analyze 28 systems and evaluate five implementations across three benchmarks, showing that existing architectures optimize for transient model constraints rather than persistent task challenges, and identifying two failure categories (Type A capability gaps and Type B complexity barriers) whose root causes require distinct solutions.
- **PENTESTGPT v2 (§4).** We present a system addressing both failure types: a Tool and Skill Layer for Type A failures, and Task Difficulty Assessment integrated into Evidence-Guided Attack Tree Search for Type B failures.
- **Evaluation across three benchmarks (§5).** PENTESTGPT v2 achieves 91% on CTF benchmarks (49% improvement), roots 12/13 machines on realistic targets, and compromises 4/5 hosts on enterprise AD, doubling baseline performance.
- **Design principles (§6).** We analyze remaining barriers (novel exploitation, adversarial robustness) and propose evaluation methodologies that separately assess Type A and Type B performance.
- **Open-source artifacts.** We release PENTESTGPT v2’s implementation, tool interfaces, and evaluation scripts to support reproducibility [3].

2 Background

2.1 Penetration Testing

Penetration testing identifies security vulnerabilities by simulating real-world attackers in blackbox/greybox scenarios. Standard methodologies decompose engagements into phases: *reconnaissance* (information gathering), *enumeration* (identifying services and entry points), *exploitation* (gaining access), and *post-exploitation* (privilege escalation and lateral movement) [26, 28]. This workflow follows a characteristic search pattern: *breadth-first exploration* over attack surfaces followed by *depth-first exploitation* along promising paths. Testers continuously decide which paths to pursue, when to abandon unproductive avenues, and how to integrate new discoveries. This interleaving of exploration and exploitation motivates our design (§4).

2.2 Benchmarking Penetration Testing

Evaluating penetration testing capabilities presents methodological challenges. Real-world engagements involve social engineering, multi-target reconnaissance, and complex business logic that cannot be easily replicated, while commercial tests produce confidential reports tied to proprietary systems. Standardized benchmarks address these con-

straints: *VulnHub* [1] provides downloadable vulnerable VMs, *HTB* [11] offers curated machines spanning difficulty levels, and *CTF* competitions present challenges across web exploitation, cryptography, and binary exploitation.

Benchmarks differ from real-world targets in important ways. CTF challenges are designed to be solvable with a single attack path, whereas real systems may have no exploitable vulnerabilities or require broad discovery across a large attack surface. GOAD (Game of Active Directory) [25] is the closest approximation to realistic enterprise environments among current benchmarks, requiring chained attack techniques across multi-domain Windows networks, though it still abstracts away social engineering and time pressure. We interpret benchmark results as measuring specific technical capabilities rather than predicting overall real-world effectiveness.

2.3 LLM-Based Agents

The standard approach for deploying LLMs as autonomous agents augments them with *tool use* [31] that invokes external functions such as shell commands or APIs, and *agentic scaffolding* that structures the interaction loop [15, 34]. Penetration testing is a natural application domain for such agents: it requires combining extensive domain knowledge with sequential decision-making, tool orchestration, and adaptive strategy. Early work explores LLMs as copilots suggesting next steps to human operators [8, 29], whereas more recent systems position LLMs as autonomous agents executing reconnaissance, exploitation, and post-exploitation workflows [17, 30, 32]. These agents must handle heterogeneous tool outputs, maintain coherent strategies across many interaction steps, and decide when to pivot between attack paths. These challenges push against the limits of current LLM capabilities. Similar limitations appear in software engineering [15] and web navigation [34], suggesting that the barriers are not specific to penetration testing.

3 Understanding LLM Agent Failures

How far are we from achieving real-world penetration testing with LLM agents? To answer this question, we conduct an empirical analysis of existing LLM-based penetration testing systems. Our goals are to (1) understand what drives reported performance improvements, (2) identify failure modes through controlled evaluation, and (3) establish a framework for distinguishing tractable tasks from intractable challenges.

3.1 Taxonomy and Evaluation of LLM-based Penetration Testing

We survey LLM-based penetration testing systems, identifying 28 candidates published between 2023–2025. Inclusion criteria require systems to use LLMs as a core component

Table 1: Taxonomy of LLM-based penetration testing systems

System	Year	Arch.	Tools	Know.	Planning
PentestGPT [8]	2024	Workflow	Shell	Prompt	Task tree
AutoPT [32]	2024	Single	Shell	Prompt	State mach.
RapidPen [23]	2025	Single	Shell	RAG	ReAct
PentestAgent [30]	2024	Multi	Func.	RAG	Phase
VulnBot [17]	2025	Multi	Shell	Prompt	Tri-phase
xOffense [19]	2024	Multi	Shell	Fine-tune	Multi-phase
TermiAgent [20]	2024	Multi	Shell	RAG	Mem. tree
Cochise [12]	2025	Multi	Shell	Prompt	Hierarchical

and target penetration testing or CTF challenges; we exclude vulnerability detection without exploitation and commercial systems without published details. Of 28 candidates, 10 meet our criteria, with the list in Appendix A.

3.1.1 Taxonomy

We summarize each system along four dimensions: *architecture* (multi-agent, human-in-the-loop), *tool integration* (function calls, MCP [5]), *knowledge sources* (Retrieval-Augmented Generation (RAG), fine-tuned), and *planning* (reactive, task trees, state machines, memory trees). Table 1 summarizes representative systems across three architectural families: human-in-the-loop copilots like PentestGPT [8], single-agent systems like AutoPT [32], and multi-agent systems like PentestAgent [30], VulnBot [17], and Cochise [12].

3.1.2 Evaluation Setup

We evaluate five representative open-source systems: **PentestGPT** [8] (copilot), **AutoPT** [32] (single-agent), **PentestAgent** [30] (multi-agent with RAG), **VulnBot** [17] (multi-agent tri-phase), and **Cochise** [12] (Active Directory (AD)-focused). Benchmarks span three realism levels: **XBOW** [2] (104 web challenges: SQL injection (SQLi), cross-site scripting (XSS), auth bypass), **PentestGPT Benchmark** [8] (13 machines from HTB and VulnHub requiring end-to-end penetration testing), and **GOAD** [25] (5-host multi-domain AD requiring chained attacks).

For each system-benchmark pair, we evaluate with GPT-4o, GPT-5, Gemini-3-Flash, and Claude Sonnet 4 to assess model vs. architecture contributions. We include GPT-4o (the model generation most existing systems were optimized for) alongside newer models to examine how architectural advantages evolve as underlying capabilities improve. §5 evaluates PENTESTGPT v2 with a different model set (GPT-5.2, Opus 4.5, Gemini 3 Pro) selected specifically for thinking mode support, enabling controlled comparison of extended reasoning. We set temperature to zero and report best-of-three trials following prior work [8, 9], since penetration testing is inherently non-deterministic.

3.2 Findings

Table 2 summarizes task completion rates across all system-model-benchmark combinations. We provide in-depth experimental results analysis below.

3.2.1 Agent Architecture Convergence

Despite two years of agent design innovation, performance differences between systems compress with state-of-the-art models. On XBOW with GPT-4o, completion rates range from 27% to 39% across five systems, a 44% relative spread that reflects meaningful architectural distinctions. With GPT-5, this gap narrows to 22.5% (40–49%); similar convergence appears on the PentestGPT Benchmark, where the spread shrinks from 2 points with GPT-4o (4–6 machines) to 1 point with GPT-5 (7–8 machines).

This convergence points to a limitation in how existing agents were designed: they address *transient* model constraints rather than *persistent* task challenges. Consider the techniques these systems employ. PentestGPT’s summarization module compensates for limited context windows, a constraint that largely dissolves as models gain native million-token support. Multi-agent architectures with role separation (e.g., reconnaissance agent, exploitation agent) work around weak instruction-following, yet frontier models handle complex multi-step prompts without explicit decomposition. RAG pipelines for tool documentation address poor parametric knowledge of security tools, yet recent models have much stronger baseline knowledge of common exploitation techniques and penetration testing tools. These “innovations” are workarounds for 2023-era model limitations, not solutions to persistent penetration testing challenges.

What distinguishes transient from persistent challenges? Transient challenges diminish as models improve: context capacity, instruction adherence, tool-use reliability, and domain knowledge all scale with model capability. Persistent challenges, by contrast, remain regardless of raw model power: long-horizon planning across 10+ exploitation steps, principled exploration-exploitation decisions, maintaining state external to degrading context, and real-time assessment of task difficulty. These challenges arise from the *structure of penetration testing tasks*, not from model limitations, and thus require architectural solutions that *complement* rather than *compensate for* underlying models.

The Cochise case shows this distinction from a different angle. Cochise’s AD-specific attack primitives (Kerberoasting, NTLM relay, BloodHound integration) are capability additions that models cannot replicate through improved reasoning alone. However, this specialization comes at the cost of generality: Cochise underperforms on XBOW and the PentestGPT Benchmark (34% and 4/13 with GPT-4o) compared to general-purpose systems like VulnBot (39% and 6/13), while leading on GOAD by leveraging domain-specific knowledge unavailable to other systems. Neither approach, compensating

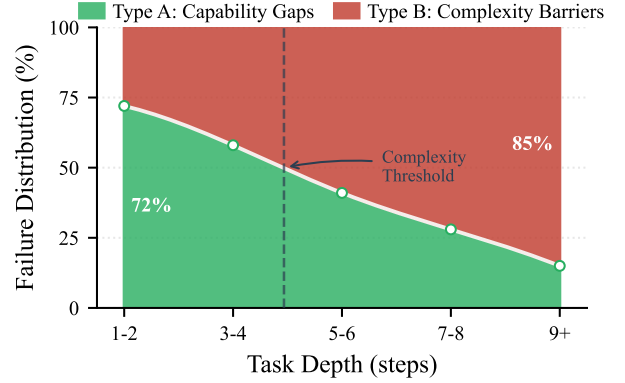


Figure 1: Failure type distribution by the task depth, measured as the number of distinct exploitation steps required for task completion.

for model limitations nor adding domain-specific capabilities, addresses the persistent challenge of navigating complex attack graphs.

Finding 1: Existing penetration testing agents address transient model limitations rather than persistent task challenges. As models evolve, benefits brought by architectural distinctions compress. Durable agent value should address challenges that persist across model evolution.

3.2.2 Two Distinct Failure Categories

To understand *why* systems fail rather than merely *how often*, we analyze 200 execution traces from unsuccessful attempts (40 per system), sampling proportionally across benchmarks. Two researchers independently coded failure modes using open coding, then reconciled disagreements through discussion. Our analysis shows that failures partition into two distinct categories, classified based on observable trace characteristics *before* any intervention.

Type A failures (capability gaps) are identified when the trace shows the agent correctly reasons about the attack vector but fails at execution: the agent articulates the correct approach but then issues malformed commands or uses incorrect tool syntax. For instance, an agent may correctly identify a SQL injection vulnerability (e.g., “I will use SQL injection to extract data”) but fail because it lacks `sqlmap` or the correct documentation. To validate this classification, we augment PentestGPT with missing tool documentation and usage instructions; XBOW completion improves from 27% to 38%, a 41% relative improvement that confirms Type A failures respond to capability engineering as predicted.

Type B failures (complexity barriers) are identified when the trace shows the agent possesses adequate tools and knowledge (evidenced by successful tool invocations earlier in the session) but fails to navigate the task space effectively. We

Table 2: Task completion rates across systems, models, and benchmarks. XBOW: task completion (%); PentestGPT Benchmark: machines rooted (/13); GOAD: hosts compromised (/5).

System	XBOW (104 tasks)				PentestGPT-Ben (13 machines)				GOAD (5 targets)			
	GPT-4o	GPT-5	Gem.	Claude	GPT-4o	GPT-5	Gem.	Claude	GPT-4o	GPT-5	Gem.	Claude
PentestGPT	27	42	36	39	5	7	6	6	0	1	1	1
AutoPT	28	40	35	37	4	7	6	6	0	1	0	0
PentestAgent	34	49	42	46	6	7	6	6	0	1	0	1
VulnBot	39	45	44	46	6	8	6	7	0	1	0	1
Cochise	34	43	39	39	4	4	4	4	1	2	2	2

Table 3: Failure mode analysis (200 traces). Type A failures resolve with tooling; Type B persist regardless.

Failure Category	Freq. (%)	Tooling?
<i>Type A: Capability Gaps (42% total)</i>		
Missing tool / Incorrect syntax	26	✓
Output parsing / Knowledge gap	16	✓
<i>Type B: Complexity Barriers (58% total)</i>		
Context forgetting	18	—
Premature commitment	16	—
Exploration-exploitation imbalance	12	—
Multi-step chain failures	12	—

identify three recurring patterns from trace analysis. *Context forgetting* occurs when credentials discovered during reconnaissance are lost by the time exploitation begins, forcing redundant discovery or causing authentication failures. *Premature commitment* occurs when agents dive deep into a single attack path without adequate reconnaissance, missing easier alternatives. *Exploration-exploitation imbalance* is the inverse: exhaustive reconnaissance that never transitions to exploitation, accumulating information without acting on it. These issues cascade into chain errors: agents complete individual attack stages successfully but fail to integrate them into coherent attack chains, losing state between phases.

The distribution of failure types varies systematically with task complexity. On XBOW, where tasks typically require 1–3 steps, Type A failures dominate (68% of failures resolve with improved tooling). On GOAD, where successful attacks require chaining 5–10 steps across multiple hosts, Type B failures dominate (79% of failures persist regardless of tooling improvements). Figure 1 visualizes this relationship: Type A failures concentrate in short-horizon tasks while Type B failures dominate in task depth beyond 5 steps. Table 3 summarizes the failure mode distribution.

Finding 2: Failures partition into (a) *Type A: capability gaps*, i.e., missing tools and knowledge addressable through engineering, and (b) *Type B: complexity barriers*, i.e., search strategy and state management failures that persist despite adequate capabilities. These two categories require different solutions.

3.3 Analysis and Design Implications

We now present further analysis and design implications.

3.3.1 Root Cause: Missing Difficulty Assessment

Type B failures share a common root cause: agents cannot distinguish tractable from intractable tasks in real time. *Premature commitment* occurs because agents cannot estimate whether a path requires 3 or 30 steps; without this estimate, they persist on unproductive branches indefinitely. *Exploration-exploitation imbalance* occurs because agents lack metrics for when reconnaissance is sufficient; they cannot determine whether gathered evidence justifies transitioning to exploitation. *Chain failures* occur partly because agents cannot assess whether their accumulated context remains adequate for the current task; critical information may have been lost or degraded without the agent’s awareness. For example, context forgetting occurs because agents lack difficulty metrics: without tracking context load, they cannot predict when accumulated history will overwhelm the model’s effective memory, leading to silent degradation of reasoning quality.

What would difficulty assessment require in practice? We identify four measurable dimensions: *horizon estimation* (remaining steps to goal), *evidence confidence* (certainty about current state), *context load* (fraction of context window consumed), and *historical success* (past performance on similar branches). These dimensions are measurable during execution, unlike abstract “difficulty” which is only knowable post-hoc. An agent that tracks these signals can decide when to persist, when to pivot, and when to prune.

Current systems uniformly lack this capability. PentestGPT’s Penetration Testing Tree (PTT) tracks attack structure but provides no difficulty metrics to guide search. AutoPT’s Pentesting State Machine (PSM) enforces phase transitions